

ADVANCED WEB TECHNOLOGY

UNIT III

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is `getElementById()`. The example below "finds" an HTML element (with `id="demo"`), and changes the element content (`innerHTML`) to "Hello JavaScript":

```
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change HTML content.</p>
<button type="button" onclick='document.getElementById("demo").innerHTML =
"Hello JavaScript!">Click Me!</button>
```

The `<script>` Tag

In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.

```
<h2>JavaScript in Body</h2>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

JavaScript in `<head>`

In this example, a JavaScript function is placed in the `<head>` section of an HTML page.

The function is invoked (called) when a button is clicked:

```
<head>
<script>
function myFunction() {
document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>JavaScript in Head</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
```

JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.

- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

Using innerHTML

To access an HTML element, JavaScript can use the `document.getElementById(id)` method. The `id` attribute defines the HTML element. The `innerHTML` property defines the HTML content:

```
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>
```

Using document.write()

For testing purposes, it is convenient to use `document.write()`:

```
<script>
document.write(5 + 6);
</script>
```

Using window.alert()

You can use an alert box to display data:

```
<script>
window.alert(5 + 6);
</script>
```

Using console.log()

For debugging purposes, you can call the `console.log()` method in the browser to display data.

```
<script>
console.log(5 + 6);
</script>
```

JavaScript Print

JavaScript does not have any print object or print methods. You cannot access output devices from JavaScript.

```
<button onclick="window.print()">Print this page</button>
```

JavaScript Statements

JavaScript statements are composed of Values, Operators, Expressions, Keywords, and Comments.

```
document.getElementById("demo").innerHTML = "Hello Dolly.";
```

Semicolons ;

Semicolons separate JavaScript statements. Add a semicolon at the end of each executable statement:

```
var a, b, c; // Declare 3 variables
a = 5;      // Assign the value 5 to a
b = 6;      // Assign the value 6 to b
c = a + b;   // Assign the sum of a and b to c
```

JavaScript White Space

JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.

```
var person = "Hege";
var person="Hege";
```

JavaScript Code Blocks

JavaScript statements can be grouped together in code blocks, inside curly brackets {...}. The purpose of code blocks is to define statements to be executed together.

```
function myFunction() {
  document.getElementById("demo1").innerHTML = "Hello Dolly!";
  document.getElementById("demo2").innerHTML = "How are you?";
}
```

JavaScript Variables

In a programming language, variables are used to store data values. JavaScript uses the var keyword **to** declare variables. An equal sign is used to assign values to variables.

```
<script>
var x;
x = 6;
document.getElementById("demo").innerHTML = x;
</script>
```

JavaScript Comments

Not all JavaScript statements are "executed". **Code after double slashes // or between /* and */ is treated as a comment.** Comments are ignored, and will not be executed:

```
<script>
var x;
x = 5;
// x = 6; I will not be executed
document.getElementById("demo").innerHTML = x;
</script>
```

JavaScript Identifiers

All JavaScript variables must be identified with unique names. These unique names are called identifiers.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter
- Names can also begin with \$ and _ (but we will not use it in this tutorial)
- Names are case sensitive (y and Y are different variables)
- Reserved words (like JavaScript keywords) cannot be used as names

JavaScript is Case Sensitive

All JavaScript identifiers are case sensitive. The variables lastName and lastname, are two different variables:

```
<script>
var lastname, lastName;
lastName = "Doe";
lastname = "Peterson";
document.getElementById("demo").innerHTML = lastName;
</script>
```

JavaScript Data Types

JavaScript variables can hold numbers like 100 and text values like "John Doe". In programming, text values are called text strings. JavaScript can handle many types of data, but for now, just think of numbers and strings. Strings are written inside double or single quotes. Numbers are written without quotes. If you put a number in quotes, it will be treated as a text string.

```
<script>
var pi = 3.14;
var person = "John Doe";
var answer = 'Yes I am!';
document.getElementById("demo").innerHTML =
pi + "<br>" + person + "<br>" + answer;
</script>
```

Value = undefined

In computer programs, variables are often declared without a value. The value can be something that has to be calculated, or something that will be provided later, like user input. A variable declared without a value will have the value undefined. The variable carName will have the value undefined after the execution of this statement:

```
<script>
```

```
var carName;  
document.getElementById("demo").innerHTML = carName;  
</script>
```

Re-Declaring JavaScript Variables

If you re-declare a JavaScript variable, it will not lose its value. The variable carName will still have the value "Volvo" after the execution of these statements:

```
<script>  
var carName = "Volvo";  
var carName;  
document.getElementById("demo").innerHTML = carName;  
</script>
```

JavaScript Arithmetic

As with algebra, you can do arithmetic with JavaScript variables, using operators like = and +:

```
<script>  
var x = 5 + 2 + 3;  
document.getElementById("demo").innerHTML = x;  
</script>
```

```
//Answer:  
10
```

```
<script>  
x = "5" + 2 + 3;  
document.getElementById("demo").innerHTML = x;  
</script>
```

```
//Answer:  
523
```

```
<script>  
var x = 2 + 3 + "5";  
document.getElementById("demo").innerHTML = x;  
</script>
```

```
//Answer:  
55
```

JavaScript Dollar Sign \$

Remember that JavaScript identifiers (names) must begin with:

- A letter (A-Z or a-z)

- A dollar sign (\$)
- Or an underscore (_)

Since JavaScript treats a dollar sign as a letter, identifiers containing \$ are valid variable names:

```
<script>
var $ = 2;
var $myMoney = 5;
document.getElementById("demo").innerHTML = $ + $myMoney;
</script>
```

Answer:

7

JavaScript Underscore (_)

Since JavaScript treats underscore as a letter, identifiers containing _ are valid variable names:

```
<script>
var _x = 2;
var _100 = 5;
document.getElementById("demo").innerHTML = _x + _100;
</script>
```

Global Scope

Variables declared Globally (outside any function) have Global Scope.

```
<script>
var carName = "Volvo";
myFunction();
function myFunction() {
document.getElementById("demo").innerHTML = "I can display " + carName;
}
</script>
```

Function Scope

Variables declared Locally (inside a function) have Function Scope.

```
<script>
myFunction();
function myFunction() {
var carName = "Volvo";
document.getElementById("demo1").innerHTML = typeof carName + " " + carName;
}
document.getElementById("demo2").innerHTML = typeof carName;
</script>
```

JavaScript Block Scope

- Variables declared with the var keyword cannot have Block Scope.
- Variables declared inside a block {} can be accessed from outside the block.
- Variables declared with the let keyword can have Block Scope.
- Variables declared inside a block {} cannot be accessed from outside the block:

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

JavaScript Global Variable

A **JavaScript global variable** is declared outside the function or declared with window object. It can be accessed from any function.

```
<script>  
var value=50;//global variable  
function a(){  
  alert(value);  
}  
function b(){  
  alert(value);  
}  
</script>
```

JavaScript Data Types

JavaScript provides different data types to hold different types of values. There are two types of data types in JavaScript.

1. Primitive data type
2. Non-primitive (reference) data type

JavaScript is a dynamic type language, means you don't need to specify type of the variable because it is dynamically used by JavaScript engine. You need to use var here to specify the data type. It can hold any type of values such as numbers, strings etc.

JavaScript primitive data types

There are five types of primitive data types in JavaScript. They are as follows:

Data Type	Description
String	represents sequence of characters e.g. "hello"
Number	represents numeric values e.g. 100
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null i.e. no value at all

- **JavaScript non-primitive data types**

The non-primitive data types are as follows:

Data Type	Description
Object	represents instance through which we can access members
Array	represents group of similar values
RegExp	represents regular expression

JavaScript Operators

JavaScript operators are symbols that are used to perform operations on operands. For example:

```
var sum=10+20;
```

Here, + is the arithmetic operator and = is the assignment operator.

There are following types of operators in JavaScript.

1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Bitwise Operators
4. Logical Operators
5. Assignment Operators
6. Special Operators

JavaScript Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations on the operands. The following operators are known as JavaScript arithmetic operators.

Operator	Description	Example
+	Addition	10+20 = 30
-	Subtraction	20-10 = 10
*	Multiplication	10*20 = 200
/	Division	20/10 = 2
%	Modulus (Remainder)	20%10 = 0
++	Increment	var a=10; a++; Now a = 11
--	Decrement	var a=10; a--; Now a = 9

JavaScript Comparison Operators

The JavaScript comparison operator compares the two operands. The comparison operators are as follows:

Operator	Description	Example
==	Is equal to	10==20 = false
===	Identical (equal and of same type)	10==20 = false
!=	Not equal to	10!=20 = true
!==	Not Identical	20!==20 = false
>	Greater than	20>10 = true
>=	Greater than or equal to	20>=10 = true
<	Less than	20<10 = false
<=	Less than or equal to	20<=10 = false

JavaScript Bitwise Operators

The bitwise operators perform bitwise operations on operands. The bitwise operators are as follows:

Operator	Description	Example
&	Bitwise AND	(10==20 & 20==33) = false
	Bitwise OR	(10==20 20==33) = false
^	Bitwise XOR	(10==20 ^ 20==33) = false
~	Bitwise NOT	(~10) = -10
<<	Bitwise Left Shift	(10<<2) = 40
>>	Bitwise Right Shift	(10>>2) = 2
>>>	Bitwise Right Shift with Zero	(10>>>2) = 2

JavaScript Logical Operators

The following operators are known as JavaScript logical operators.

Operator	Description	Example
&&	Logical AND	(10==20 && 20==33) = false
	Logical OR	(10==20 20==33) = false
!	Logical Not	!(10==20) = true

JavaScript Assignment Operators

The following operators are known as JavaScript assignment operators.

Operator	Description	Example
=	Assign	10+10 = 20
+=	Add and assign	var a=10; a+=20; Now a = 30
-=	Subtract and assign	var a=20; a-=10; Now a = 10
=	Multiply and assign	var a=10; a=20; Now a = 200
/=	Divide and assign	var a=10; a/=2; Now a = 5
%=	Modulus and assign	var a=10; a%=2; Now a = 0

JavaScript Special Operators

The following operators are known as JavaScript special operators.

Operator	Description
(?:)	Conditional Operator returns value based on the condition. It is like if-else.
,	Comma Operator allows multiple expressions to be evaluated as single statement.
delete	Delete Operator deletes a property from the object.
in	In Operator checks if object has the given property
instanceof	checks if the object is an instance of given type
new	creates an instance (object)
typeof	checks the type of object.
void	it discards the expression's return value.

JavaScript If-else

The **JavaScript if-else statement** is used *to execute the code whether condition is true or false*.

There are three forms of if statement in JavaScript.

1. If Statement
2. If else statement
3. if else if statement

JavaScript If statement

It evaluates the content only if expression is true. The signature of JavaScript if statement is given below.

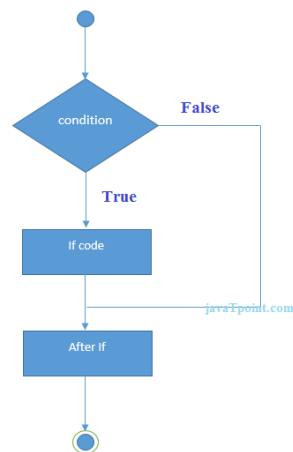
Syntax:

```
if(expression){  
  //content to be evaluated  
}
```

Example:

```
<script>  
var a=20;  
if(a>10){  
  document.write("value of a is greater than 10");  
}
```

```
}  
</script>
```



Flowchart of JavaScript If statement

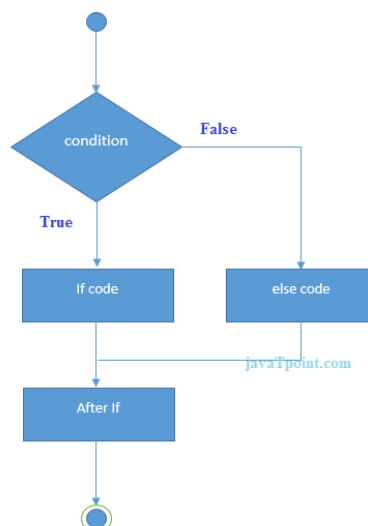
JavaScript If...else Statement

It evaluates the content whether condition is true or false. The syntax of JavaScript if-else statement is given below.

Syntax:

```
if(expression){  
  //content to be evaluated if condition is true  
}  
else{  
  //content to be evaluated if condition is false  
}
```

Flowchart of JavaScript If...else statement



Example:

```
<script>  
var a=20;
```

```
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
```

JavaScript If...else if statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

Syntax:

```
if(expression1){
//content to be evaluated if expression1 is true
}
else if(expression2){
//content to be evaluated if expression2 is true
}
else if(expression3){
//content to be evaluated if expression3 is true
}
else{
//content to be evaluated if no expression is true
}
```

Example:

```
<script>
var a=20;
if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
```

</script>

JavaScript Switch

The **JavaScript switch statement** is used *to execute one code from multiple expressions*. It is just like else if statement that we have learned in previous page. But it is convenient than *if..else..if* because it can be used with numbers, characters etc.

Syntax:

```
switch(expression){  
  case value1:  
    code to be executed;  
    break;  
  case value2:  
    code to be executed;  
    break;  
  .....  
  
  default:  
    code to be executed if above values are not matched;  
}
```

Example:

```
<script>  
var grade='B';  
var result;  
switch(grade){  
  case 'A':  
    result="A Grade";  
    break;  
  case 'B':  
    result="B Grade";  
    break;  
  case 'C':  
    result="C Grade";  
    break;  
  default:  
    result="No Grade";  
}  
document.write(result);  
</script>
```

JavaScript Loops

The JavaScript loops are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

There are four types of loops in JavaScript.

1. for loop
2. while loop
3. do-while loop
4. for-in loop

for loop

The JavaScript for loop *iterates the elements* for the fixed number of times. It should be used if number of iteration is known. The syntax of for loop is given below.

Syntax:

```
for (initialization; condition; increment)
{
    code to be executed
}
```

Example

```
<script>
for (i=1; i<=5; i++)
{
    document.write(i + "<br/>")
}
</script>
```

while loop

The JavaScript while loop iterates the elements for the infinite *number of times*. It should be used if number of iteration is not known. The syntax of while loop is given below.

Syntax:

```
<script>
while (condition)
{
    code to be executed
}
```

Example:

```
<script>
var i=11;
while (i<=15)
{
    document.write(i + "<br/>");
    i++;
}
```

</script>

do while loop

The JavaScript do while loop iterates the elements for the infinite number of times like while loop. But, code is *executed at least* once whether condition is true or false. The syntax of do while loop is given below.

Syntax:

```
do{
    code to be executed
}while (condition);
```

```
<script>
var i=21;
do{
    document.write(i + "<br/>");
    i++;
}while (i<=25);
</script>
```

Objects

A JavaScript object is an entity having state and behavior (properties and method). For example: car, pen, bike, chair, glass, keyboard, monitor etc. JavaScript is an object-based language. Everything is an object in JavaScript.

Creating Objects

There are 3 ways to create objects.

1. By object literal
2. By creating instance of Object directly (using new keyword)
3. By using an object constructor (using new keyword)

Object by object literal

Syntax:

```
object={property1:value1,property2:value2.....propertyN:valueN}
```

Example:

```
<script>
emp={id:102,name:"Shyam Kumar",salary:40000}
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
```

By creating instance of Object

Syntax:


```
var objectname=new Object();
```

Example

```
<script>
var emp=new Object();
emp.id=101;
emp.name="Ravi Malik";
emp.salary=50000;
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
```

By using an Object constructor

Here, you need to create function with arguments. Each argument value can be assigned in the current object by using this keyword.

Example:

```
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;
}
e=new emp(103,"Vimal Jaiswal",30000);
document.write(e.id+" "+e.name+" "+e.salary);
</script>
```

JavaScript Array

JavaScript array is an object that represents a collection of similar type of elements.

There are 3 ways to construct array in JavaScript

1. By array literal
2. By creating instance of Array directly (using new keyword)
3. By using an Array constructor (using new keyword)

JavaScript array literal

Example:

```
var arrayname=[value1,value2.....valueN];
<script>
var emp=["Sonoo","Vimal","Ratan"];
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br/>");
}
</script>
```

JavaScript Array directly (new keyword)

Syntax:

```
var arrayname=new Array();
```

Example:

```
<script>
var i;
var emp = new Array();
emp[0] = "Arun";
emp[1] = "Varun";
emp[2] = "John";

for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
```

JavaScript array constructor (new keyword)

Here, you need to create instance of array by passing arguments in constructor so that we don't have to provide value explicitly.

Example:

```
<script>
var emp=new Array("Jai","Vijay","Smith");
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
```

JavaScript Functions

JavaScript functions are used to perform operations. We can call JavaScript function many times to reuse the code.

Advantage of JavaScript function

There are mainly two advantages of JavaScript functions.

- Code reusability: We can call a function several times so it save coding.
- Less coding: It makes our program compact. We don't need to write many lines of code each time to perform a common task.

Syntax:

```
function functionName([arg1, arg2, ...argN]){
//code to be executed
}
```

Example:

```

<script>
function msg(){
alert("hello! this is message");
}
</script>
<input type="button" onclick="msg()" value="call function"/>

```

JavaScript Function Arguments

We can call function by passing arguments. Let's see the example of function that has one argument.

Example:

```

<script>
function getcube(number){
alert(number*number*number);
}
</script>
<form>
<input type="button" value="click" onclick="getcube(4)"/>
</form>

```

Function with Return Value

We can call function that returns a value and use it in our program. Let's see the example of function that returns value.

Example:

```

<script>
function getInfo(){
return "hello javatpoint! How r u?";
}
</script>
<script>
document.write(getInfo());
</script>

```

- **Callback Function**

A callback is a function passed as an argument to another function. Using a callback, you could call the calculator function (myCalculator) with a callback, and let the calculator function run the callback after the calculation is finished:

Example:

```

function myDisplayer(some) {
    document.getElementById("demo").innerHTML = some;
}

```

```
function myCalculator(num1, num2, myCallback) {
  let sum = num1 + num2;
  myCallback(sum);
}
myCalculator(5, 5, myDisplayer);
```

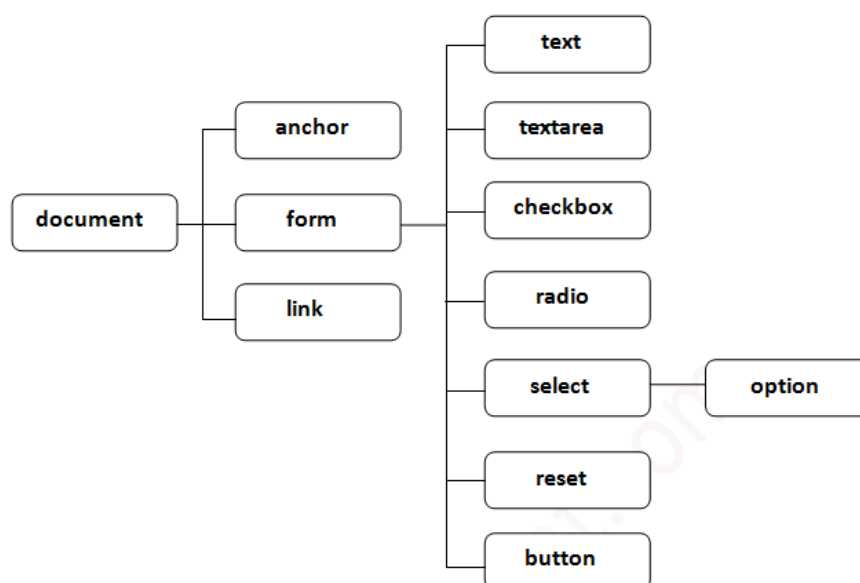
Where callbacks really shine are in asynchronous functions, where one function has to wait for another function (like waiting for a file to load).

Document Object Model (DOM)

The document object represents the whole html document. When html document is loaded in the browser, it becomes a document object. It is the **root element** that represents the html document. It has properties and methods. By the help of document object, we can add dynamic content to our web page.

Properties of document object

Let's see the properties of document object that can be accessed and modified by the document object.



- **Accessing field value by document object**

In this example, we are going to get the value of input text by user. Here, we are using `document.form1.name.value` to get the value of name field. Here, `document` is the root element that represents the html document. Where,

`form1` is the name of the form.

`name` is the attribute name of the input text.

Example:

```
<script type="text/javascript">
function printvalue(){
var name=document.form1.name.value;
alert("Welcome: "+name);
}
</script>

<form name="form1">
Enter Name:<input type="text" name="name"/>
<input type="button" onclick="printvalue()" value="print name"/>
</form>
```

AJAX

AJAX = Asynchronous JavaScript And XML.

AJAX is not a programming language.

AJAX just uses a combination of:

- A browser built-in XMLHttpRequest object (to request data from a web server)
- JavaScript and HTML DOM (to display or use the data)

AJAX is a developer's dream, because you can:

- Update a web page without reloading the page
- Request data from a server - after the page has loaded
- Receive data from a server - after the page has loaded
- Send data to a server - in the background

Example:

```
<!DOCTYPE html>
<html>
<body>
<div id="demo">
  <h2>Let AJAX change this text</h2>
  <button type="button" onclick="loadDoc()">Change Content</button>
</div>
</body>
</html>
```

The HTML page contains a <div> section and a <button>.

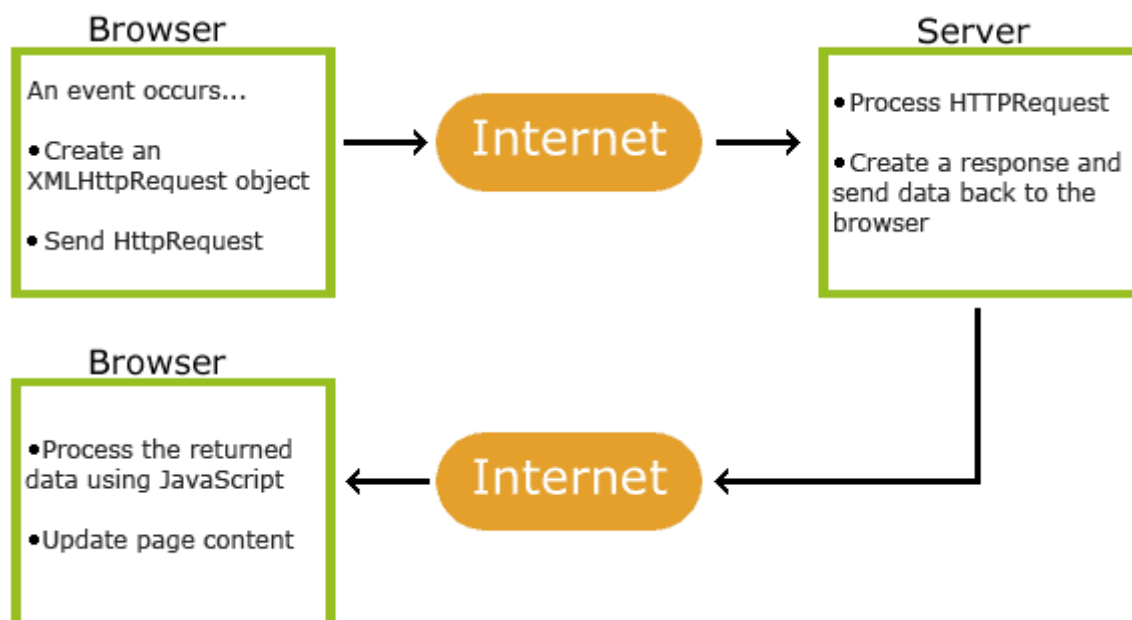
The <div> section is used to display information from a server.

The <button> calls a function (if it is clicked).

The function requests data from a web server and displays it:

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}
```

How AJAX Works



- An event occurs in a web page (the page is loaded, a button is clicked)
 - An XMLHttpRequest object is created by JavaScript
 - The XMLHttpRequest object sends a request to a web server
 - The server processes the request
 - The server sends a response back to the web page
 - The response is read by JavaScript
-
- Proper action (like page update) is performed by JavaScript

Why is Ajax Used?

Ajax is a web developer's long term dream because the user can do the following things:

- Without reloading the page, the user request can be updated
- After the page is loaded, it generates data from the server.
- Receive data from the server after the page has loaded.
- In the background, sends data to the server

• Advantages and Disadvantages of Ajax

- Every software has its pros and cons, which gives good results when handled in a correct way.

Advantages

- Reduces the server traffic and increases the speed
- It is responsive, and the time taken is also less
- Form validation
- Bandwidth usage can be reduced
- Asynchronous calls can be made; this reduces the time for data arrival.

Disadvantages

- Open-source
- Active x request is created only in internet explorer and a newly created web browser.
- For security reason, you can only access information from the web host that serves pages. Fetching information from other servers is not possible with Ajax.

Examples of Ajax Application

Given below are the lists of web applications that commonly use Ajax

- Google suggests that auto-complete options will be offered while typing when a user enters the search query in the Google search bar. Suggestions given by Google can be navigated by using operational keys.
- Yahoo maps are easier while operating, and user experiences more fun. This Map uses Ajax to drag the entire map with the mouse without using buttons that will be at ease to the user.
- Google maps are general applications that use Ajax. This is a real-time application in which the user can manipulate the data and change the view settings. Ajax directly works on a web browser without any plugin installations. Firstly, only Microsoft internet explorer used Ajax, but due to its reliability, more web applications like chrome, Mozilla... Etc. using this.

What is jQuery?

- jQuery is a lightweight, "write less, do more", JavaScript library.
- The purpose of jQuery is to make it much easier to use JavaScript on your website.
- jQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish, and wraps them into methods that you can call with a single line of code.
- jQuery also simplifies a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation.

The jQuery library contains the following features:

- HTML/DOM manipulation
- CSS manipulation
- HTML event methods
- Effects and animations
- AJAX
- Utilities

Why jQuery?

There are lots of other JavaScript libraries out there, but jQuery is probably the most popular, and also the most extendable. Many of the biggest companies on the Web use jQuery, such as:

- Google
- Microsoft
- IBM
- Netflix

Adding jQuery to Your Web Pages

There are several ways to start using jQuery on your web site. You can:

- Download the jQuery library from [jQuery.com](https://jquery.com)
- Include jQuery from a CDN, like Google

Downloading jQuery

There are two versions of jQuery available for downloading:

- Production version - this is for your live website because it has been minified and compressed
- Development version - this is for testing and development (uncompressed and readable code)

Both versions can be downloaded from [jQuery.com](https://jquery.com).

```
<head>
<script src="jquery-3.5.1.min.js"></script>
</head>
```

jQuery CDN

If you don't want to download and host jQuery yourself, you can include it from a CDN (Content Delivery Network).

Google is an example of someone who host jQuery:

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
</head>
```


jQuery Syntax

The jQuery syntax is tailor-made for selecting HTML elements and performing some action on the element(s).

Basic syntax is:

`$(selector).action()`

- A \$ sign to define/access jQuery
- A (*selector*) to "query (or find)" HTML elements
- A jQuery *action()* to be performed on the element(s)

Examples:

- `$(this).hide()` - hides the current element.
- `$("p").hide()` - hides all <p> elements.
- `$(".test").hide()` - hides all elements with class="test".
- `$("#test").hide()` - hides the element with id="test".

jQuery Selectors

jQuery selectors allow you to select and manipulate HTML element(s).

Using element Selector

The jQuery element selector selects elements based on the element name.

You can select all <p> elements on a page like this:

```
$("p")
```

Example

When a user clicks on a button, all <p> elements will be hidden:

```
$(document).ready(function(){
    $("button").click(function(){
        $("p").hide();
    });
});
```

Using #id selector

```
$(document).ready(function(){
    $("button").click(function(){
        $("#test").hide();
    });
});
```

Using .class selector

```
$(document).ready(function(){
    $("button").click(function(){
        $(".test").hide();
    });
});
```